PLC 工程师交底清单：耗材搬运协议（v2.5）

> 本文档是 PLC 工程师实施 ScrapeCNC / Collect 耗材搬运的核心参考。

1. OPC UA 变量定义

1.1 PC → PLC 参数（PC 在 `<stage>_Enable` 上升沿前 ≥50ms 写入）

```
scrape_Plate_Op            : INT   # 0=NONE / 1=PUT_NEW / 2=SWAP
scrape_Fetch_Rack_Plate    : INT   # 0=忽略, 1-12=从耗材架第N板搬到暂存A
scrape_Old_Plate_Slot      : INT   # 0=忽略, 1-12=旧板归还料架原孔号（仅 op=2）
scrape_Consume_Slot        : INT   # 1-6=从暂存A第N孔取空粉末收集器

collect_Plate_Op           : INT   # 0=NONE / 1=PUT_NEW / 2=SWAP
collect_Fetch_Rack_Plate   : INT   # 0=忽略, 1-12=从耗材架第N板搬到暂存B
collect_Old_Plate_Slot     : INT   # 0=忽略, 1-12=旧板归还料架原孔号（仅 op=2）
collect_Consume_Slot       : INT   # 1-6=从暂存B第N孔取空玻璃瓶
collect_Powder_Return_Slot : INT   # 1-6=粉末收集器归还暂存A原孔号（Step 50 用）
```

类型约束：所有变量 INT16，PLC ST 中只读。

1.2 PLC → PC 状态（PC 周期读取）

```
IX11 : BYTE   # bit0..bit7 = 料库检测1..8（粉末板1-6 + 玻璃瓶板1-2）
IX12 : BYTE   # bit0..bit3 = 料库检测9..12（玻璃瓶板3-6）；bit4..bit7 保留
```

由 DI 模块写，PC 周期读后按 LSB 位序解码 12 个 bool。

1.3 动作码语义表（核心）

Op	语义	Fetch_Rack_Plate	Old_Plate_Slot	机器人动作
0 NONE	暂存有耗材，复用	0	0	跳过搬板
1 PUT_NEW	暂存空，仅放新板	1-12 新板号	0	PUT_PLATE_TO_STAGING_X(Fetch)
2 SWAP	暂存满耗尽，换板	1-12 新板号	1-12 旧板原孔	RETURN_PLATE_TO_RACK_X(Old) → PUT_PLATE_TO_STAGING_X(Fetch)

> 暂存区无传感器，PLC 完全按 PC 下发的 op 执行，不推理暂存状态。

2. FSM Step 编号

Scrape FSM

```
Step 10: init
Step 12: 动作A — 读 scrape_Plate_Op，CASE 分支：
         0 → 跳到 13
         1 → PUT_PLATE_TO_STAGING_A(Fetch)
         2 → RETURN_PLATE_TO_RACK_A(Old) → PUT_PLATE_TO_STAGING_A(Fetch)
         其它 → _Step:=90
Step 13: 动作B — TAKE_FROM_STAGING_A(Consume_Slot)，记住孔号供 Step 40
Step 15/20/25-29/30: 现有刮板逻辑
Step 40: 动作C — TRANSFER_USED_TO_COLLECT（Collect 忙则等待）
Step 80: cleanup / Step 90: error
```

Collect FSM

```
Step 10: init
Step 12: 动作A' — 读 collect_Plate_Op，CASE 同 Scrape Step 12（针对暂存B）
Step 13: 动作B' — TAKE_FROM_STAGING_B(Consume_Slot)
Step 20-29: 接续 Scrape Step 40 转移的收集器
Step 30: 实际收集
Step 40: 动作D — RETURN_USED_TO_STAGING_B(Step 13 记住的孔号)
Step 50: 动作E — RETURN_POWDER_TO_STAGING_A(collect_Powder_Return_Slot)
Step 80: cleanup / Step 90: error
```

4. 防护与边缘场景

4.1 关键防护要点

项目	要求
动作码范围校验	Plate_Op ∈ {0,1,2}，其它值 → _Step:=90
参数范围校验	Fetch ∈ [0,12]，Old ∈ [0,12]，Consume ∈ [1,6]，Powder_Return ∈ [1,6]
op-参数一致性	op=PUT_NEW 需 Fetch≥1；op=SWAP 需 Fetch≥1 且 Old≥1；op=NONE 不检查不执行
Step 10 init	内部记忆变量（含 _SwapPhase）必须清零
机器人握手	Robot_Trigger 上升沿触发，Robot_Done 后清零

项目	要求
Step 50 守卫	Powder_Return_Slot=0 → _Step:=90

4.2 边缘场景

#	场景	期望
1	scrape.enabled=FALSE 但 collect.enabled=TRUE	_Error=TRUE
2	scrape 视觉失败走安全占位 G-code	Step 40 仍正常执行（空收集器流转）
3	collect 重复执行（多 Band 循环）	每次独立 FSM 周期，Powder_Return_Slot 由 PC 重写
4	scrape_Plate_Op=99 非法值	Step 12 跳 _Step=90
5	op=PUT_NEW 但 Fetch=0（PC 异常）	Step 12 范围校验跳 _Step=90
6	op=SWAP 但 Old_Plate_Slot=0	Step 12 范围校验跳 _Step=90
7	collect_Powder_Return_Slot=0 到达 Step 50	_Error=TRUE
8	急停触发	机器人动作中止，按既有急停协议处理

5. ST 伪代码参考

5.1 GVL

```
VAR_GLOBAL
    scrape_Plate_Op            : INT := 0;
    scrape_Fetch_Rack_Plate    : INT := 0;
    scrape_Old_Plate_Slot      : INT := 0;
    scrape_Consume_Slot        : INT := 0;
    collect_Plate_Op           : INT := 0;
    collect_Fetch_Rack_Plate   : INT := 0;
    collect_Old_Plate_Slot     : INT := 0;
    collect_Consume_Slot       : INT := 0;
    collect_Powder_Return_Slot : INT := 0;
    IX11 : BYTE := 16#00;
    IX12 : BYTE := 16#00;
END_VAR
```

5.2 Scrape FSM Step 12 / 13 / 40

```
VAR
    _scrape_Op           : INT := 0;
    _scrape_FetchPlate   : INT := 0;
    _scrape_OldPlate     : INT := 0;
    _scrape_RememberSlot : INT := 0;
    _scrape_SwapPhase    : INT := 0;  (* 0=待取旧, 1=待放新 *)
END_VAR

CASE _Step OF
    10: _scrape_Op := 0; _scrape_FetchPlate := 0; _scrape_OldPlate := 0;
        _scrape_RememberSlot := 0; _scrape_SwapPhase := 0;
        _Step := 12;

    12: _scrape_Op         := scrape_Plate_Op;
        _scrape_FetchPlate := scrape_Fetch_Rack_Plate;
        _scrape_OldPlate   := scrape_Old_Plate_Slot;
        CASE _scrape_Op OF
            0:  _Step := 13;   (* NONE *)

            1:  IF _scrape_FetchPlate < 1 OR _scrape_FetchPlate > 12 THEN
                    _Step := 90; RETURN;
                END_IF
                Robot_ActionID := PUT_PLATE_TO_STAGING_A;
                Robot_Param    := _scrape_FetchPlate;
                Robot_Trigger  := TRUE;
                IF Robot_Done THEN
                    Robot_Trigger := FALSE; _Step := 13;
                END_IF

            2:  IF _scrape_FetchPlate < 1 OR _scrape_FetchPlate > 12
                   OR _scrape_OldPlate < 1 OR _scrape_OldPlate > 12 THEN
                    _Step := 90; RETURN;
                END_IF
                IF _scrape_SwapPhase = 0 THEN
                    Robot_ActionID := RETURN_PLATE_TO_RACK_A;
                    Robot_Param    := _scrape_OldPlate;
                    Robot_Trigger  := TRUE;
                    IF Robot_Done THEN
                        Robot_Trigger := FALSE; _scrape_SwapPhase := 1;
                    END_IF
                ELSE
                    Robot_ActionID := PUT_PLATE_TO_STAGING_A;
                    Robot_Param    := _scrape_FetchPlate;
                    Robot_Trigger  := TRUE;
                    IF Robot_Done THEN
                        Robot_Trigger := FALSE; _Step := 13;
                    END_IF
```

```
                END_IF

            ELSE _Step := 90;
        END_CASE

    13: _scrape_RememberSlot := scrape_Consume_Slot;
        IF _scrape_RememberSlot < 1 OR _scrape_RememberSlot > 6 THEN
            _Step := 90; RETURN;
        END_IF
        Robot_ActionID := TAKE_FROM_STAGING_A;
        Robot_Param    := _scrape_RememberSlot;
        Robot_Trigger  := TRUE;
        IF Robot_Done THEN Robot_Trigger := FALSE; _Step := 20; END_IF

    (* Step 15/20/25-29/30: 既有刮板逻辑不变 *)

    40: Robot_ActionID := TRANSFER_USED_TO_COLLECT;
        Robot_Param    := _scrape_RememberSlot;
        Robot_Trigger  := TRUE;
        IF Robot_Done THEN Robot_Trigger := FALSE; _Step := 80; END_IF
END_CASE
```

5.3 Collect FSM Step 12 / 13 / 40 / 50

```
VAR
    _collect_Op               : INT := 0;
    _collect_FetchPlate       : INT := 0;
    _collect_OldPlate         : INT := 0;
    _collect_RememberSlot     : INT := 0;
    _collect_SwapPhase        : INT := 0;
    _collect_PowderReturnSlot : INT := 0;
END_VAR

CASE _Step OF
    10: _collect_Op := 0; _collect_FetchPlate := 0; _collect_OldPlate := 0;
        _collect_RememberSlot := 0; _collect_SwapPhase := 0;
        _collect_PowderReturnSlot := 0;
        _Step := 12;

    12: _collect_Op         := collect_Plate_Op;
        _collect_FetchPlate := collect_Fetch_Rack_Plate;
        _collect_OldPlate   := collect_Old_Plate_Slot;
        CASE _collect_Op OF
            0:  _Step := 13;

            1:  IF _collect_FetchPlate < 1 OR _collect_FetchPlate > 12 THEN
                    _Step := 90; RETURN;
                END_IF
                Robot_ActionID := PUT_PLATE_TO_STAGING_B;
                Robot_Param    := _collect_FetchPlate;
                Robot_Trigger  := TRUE;
                IF Robot_Done THEN
                    Robot_Trigger := FALSE; _Step := 13;
                END_IF

            2:  IF _collect_FetchPlate < 1 OR _collect_FetchPlate > 12
                   OR _collect_OldPlate < 1 OR _collect_OldPlate > 12 THEN
                    _Step := 90; RETURN;
                END_IF
                IF _collect_SwapPhase = 0 THEN
                    Robot_ActionID := RETURN_PLATE_TO_RACK_B;
                    Robot_Param    := _collect_OldPlate;
                    Robot_Trigger  := TRUE;
                    IF Robot_Done THEN
                        Robot_Trigger := FALSE; _collect_SwapPhase := 1;
                    END_IF
                ELSE
                    Robot_ActionID := PUT_PLATE_TO_STAGING_B;
                    Robot_Param    := _collect_FetchPlate;
                    Robot_Trigger  := TRUE;
                    IF Robot_Done THEN
```

```
                    Robot_Trigger := FALSE; _Step := 13;
                END_IF
            END_IF

        ELSE _Step := 90;
    END_CASE

13: _collect_RememberSlot := collect_Consume_Slot;
    IF _collect_RememberSlot < 1 OR _collect_RememberSlot > 6 THEN
        _Step := 90; RETURN;
    END_IF
    Robot_ActionID := TAKE_FROM_STAGING_B;
    Robot_Param    := _collect_RememberSlot;
    Robot_Trigger  := TRUE;
    IF Robot_Done THEN Robot_Trigger := FALSE; _Step := 20; END_IF

(* Step 20-29: 接续上游转移；Step 30: 实际收集 *)

40: Robot_ActionID := RETURN_USED_TO_STAGING_B;
    Robot_Param    := _collect_RememberSlot;
    Robot_Trigger  := TRUE;
    IF Robot_Done THEN Robot_Trigger := FALSE; _Step := 50; END_IF

50: _collect_PowderReturnSlot := collect_Powder_Return_Slot;
    IF _collect_PowderReturnSlot < 1 OR _collect_PowderReturnSlot > 6 THEN
        _Step := 90; RETURN;
    END_IF
    Robot_ActionID := RETURN_POWDER_TO_STAGING_A;
    Robot_Param    := _collect_PowderReturnSlot;
    Robot_Trigger  := TRUE;
    IF Robot_Done THEN Robot_Trigger := FALSE; _Step := 80; END_IF
END_CASE
```

6. 联调用例（PLC 侧自检）

#	输入	期望行为
1	scrape: Op=1, Fetch=2, Old=0, Consume=1	Step 12 PUT_PLATE_TO_STAGING_A(2) → Step 13 TAKE_FROM_STAGING_A(1)
2	scrape: Op=0, Fetch=0, Old=0, Consume=4	Step 12 跳过 → Step 13 TAKE_FROM_STAGING_A(4)
3	scrape: Op=2, Fetch=3, Old=2, Consume=1	Step 12 RETURN_PLATE_TO_RACK_A(2) → PUT_PLATE_TO_STAGING_A(3) → Step 13 TAKE(1)

#	输入	期望行为
4	Step 40 时 Collect 忙	Step 40 内等待
5	collect: Powder_Return=3	Step 50 RETURN_POWDER_TO_STAGING_A(3)
6	collect: Powder_Return=0 到达 Step 50	_Error=TRUE
7	scrape_Plate_Op=99	Step 12 跳 _Step=90 _Error=TRUE
8	op=2 但 Old=0	Step 12 跳 _Step=90 _Error=TRUE
9	急停触发	机器人动作中止